

Infix to Postfix Conversion & Recursion

Q. Discuss the application of Stacks in evaluating mathematical expressions. Explain the process of Infix to Postfix conversion, Postfix evaluation, and how stacks handle Recursion.

[DEF] Definition to Remember

Infix notation ($A + B$) is human-readable but hard for computers. Postfix notation ($A B +$) places operators after operands -- no parentheses or precedence rules needed. A Stack is used to convert Infix $\rightarrow$ Postfix and to evaluate Postfix expressions. Stacks also manage Recursion through the Call Stack.

1. Operator Precedence (for conversion)

Operator	Precedence
`^` (Exponentiation)	Highest (3)
`*`, `/`	Medium (2)
`+`, `-`	Lowest (1)

2. Infix to Postfix Conversion Algorithm

1. Scan expression left to right.

2. Op

Example: $A * (B + C)$

Symbol	Stack	Output
`A`	--	`A`
`*`	`*`	`A`
`(`	`*`(`	`A`
`B`	`*`(`	`A B`
`+`	`*`(`+`	`A B`
`C`	`*`(`+`	`A B C`
)`	`*`	`A B C +`
End	--	**`A B C + *`**

3. Postfix Evaluation Algorithm

1. Scan postfix string left to right.

2. Op

Example: $5\ 3\ +\ 2\ *\ \rightarrow (5+3)*2 = 16$

4. Recursion and the Call Stack

- * When a function calls itself, the OS uses the Call Stack to manage it.

* Ea

```
factorial(3)
```

```
factorial(3) -> 3    <- top (most recent call)
```

```
factorial(2) -> 2
factorial(1) -> 1    <- base case returns first
```

[MUST-WRITE] Must-Write Points (for 10 marks)

1. Infix = human-readable; Postfix = machine-friendly (no parentheses or precedence needed).
2. Infix-to-Postfix: operand->output, operator->stack (with precedence), (->push,)->pop to output.
3. Precedence order: ^ > *, / > +, -.
4. Postfix evaluation: operand->push, operator->pop two, apply, push result.
5. Final result of postfix evaluation is at the top of the stack.
6. Recursion uses the Call Stack -- each call creates a Stack Frame (pushed/popped).
7. Stack Overflow in recursion: no base case or recursion too deep.

[RECALL] Quick Recall

Infix->Postfix (operand->output, operator->stack by precedence, brackets) -> Postfix Eval (operand->push, operator->pop 2, compute, push) -> Recursion (Call Stack -> Stack Frames -> base case pops all)